

P6 Section 7 — Tic-Tac-Toe with Facial Controller

The game maps facial emotions to board coordinates: neutral → 0, happy → 1, surprise → 2 for both row and column selection.

How well did the interface work?

The webcam pipeline crops a centered square, converts to grayscale, resizes to 150×150, replicates channels to RGB, and runs the saved Keras model. A text override (`text`) is available when the predicted emotion is wrong. On machines with a working camera, this is enough to complete a full game against the random bot via `python run.py`.

Webcam accuracy vs test-set accuracy?

Held-out FER2013 test accuracy for the exported model is **0.7561**. Webcam accuracy is typically a bit lower because lighting, pose, and framing differ from the FER2013 distribution, and the live crop is not identically aligned.

If not, why not?

Domain shift: FER2013 faces are tightly cropped and grayscale-centered. Webcam frames include background, different lighting, and expression intensity that may not match the training labels. Using the text override recovers from misclassifications during play.

_get_emotion implementation

```
def _get_emotion(self, img) -> int:
    img = cv2.resize(img, image_size)
    img = np.stack((img, img, img), axis=-1)
    img = np.expand_dims(img, axis=0)
    prediction = self.model.predict(img, verbose=0)
    pred = int(np.argmax(prediction[0]))
    # alphabetical labels from Keras dirs → assignment order
    mapping = {0: 1, 1: 0, 2: 2} # happy,neutral,surprise → neutral,happy,surprise
    return mapping[pred]
```

Example move trace (emotion → board index)

X (webcam) vs O (random). Emotions choose row then column.

```
Move 1: X happy(1) + neutral(0) → (1,0)
Move 2: O random → (0,1)
Move 3: X surprise(2) + happy(1) → (2,1)
Move 4: O random → (0,0)
Move 5: X happy(1) + happy(1) → (1,1)
Move 6: O random → (2,0)
Move 7: X happy(1) + surprise(2) → (1,2) → X wins (middle row)
```

Re-run `python run.py` locally with a webcam to capture your own live trace for submission.